

Student: Malibekov Iliyas (230103344)

Date: 18.09.2026

Repository: <https://github.com/IliyasMali/Parallel-Comp>

Part 1: The Phantom Bug

Run	hits	pi number	time
1	11,939,028	0.955122	93 ms
2	11,883,966	0.950717	89 ms
3	11,758,461	0.940677	89 ms
4	11,814,606	0.945168	88 ms
5	11,909,952	0.952796	91 ms

Part 2: The Synchronization Trap

Variant	pi number	time	vs. single thread
Single thread (plain loop)	3.141590	118 ms	1.00x
synchronized (4 threads)	3.141404	1654 ms	14.02x slower
AtomicLong (4 threads)	3.141636	926 ms	7.85x slower

Part 3: OpenMP-Style Reduction

Threads (T)	Runtime (ms)	pi number	Speedup vs. 1 Thread	Efficiency
1 (baseline)	223	3.141427	1.00x	100%
2	118	3.141508	1.89x	94.5%
4	65	3.141530	3.43x	85.8%
8	52	3.141445	4.29x	53.6%
16	52	3.141865	4.29x	26.8%
32	49	3.142108	4.55x	14.2%

Question 1 (16 threads not faster than 8)

You have 8 cores. With 8 threads, each gets its own core. With 16 threads, there aren't enough cores, so they take turns on the same 8. The amount of work is the same, plus the cost of switching between threads, so it comes out to 52 ms both times. Also, 4 of the 8 cores are slow E-cores, so the speedup already starts slowing down between 4 and 8 threads.

Question 2 (synchronized slower than one core)

One thread keeps the counter in a register and never touches memory, so it's fast: 118 ms. synchronized makes all 4 threads grab a lock on every ++, about 39 million times. The lock lives in one cache line, which keeps jumping between cores, and if the lock is busy the thread gets put to sleep by the OS, which is very expensive. The useful work is one addition, the overhead is hundreds of cycles, so the cores wait in line instead of working in parallel. That's the 1654 ms. Part 3 with no locks runs in 52 ms, which proves the point.